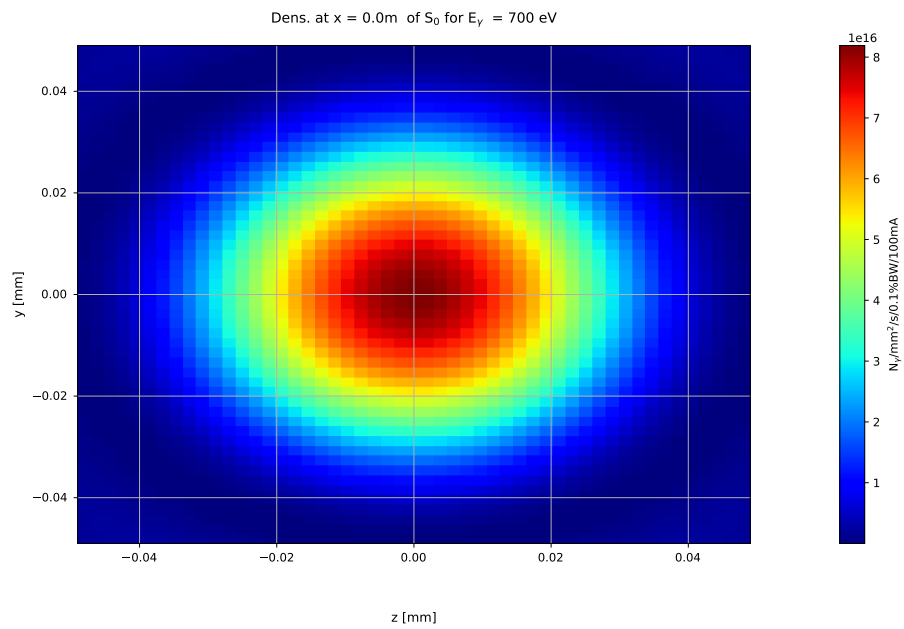


PyBrill Tutorial

Michael Scheer
October 2, 2026



PyBrill has been developed at BESSY for the fast and easy calculation of spontaneous synchrotron radiation of an analytical undulator model. This includes brilliance, radiation field, Stokes parameters, Wigner-Distributions. Monte-Carlo techniques allow to take emittance and energy-spread of the electron beam into account and to generate photons for ray-tracing.

PyBrill and the related software is published under the GNU General Public License. To get pyBrill, please contact the author or get it from github:

git clone <https://github.com/MicScheer/brill>.git brill

Contents

I	Requirements	2
I.1	Python	2
I.2	FORTRAN	3
II	Getting Started	5
II.1	Brilliance Calculations	6
II.2	Spectrum Calculations	7
III	Examples	9
III.1	Basic Beam Electron Monte-Carlo	9
III.2	Combining Beam Electron and Observation Point Monte-Carlo	12
III.3	Back Propagation of the Radiation to the Source Plane	14
III.4	Generating Photons from Undulator Source for Ray-tracing	16
III.5	Wigner Distribution	17

I Requirements

I.1 Python

To run *pyBrill* python3 must be installed on your machine. Within this tutorial a *Linux* machine is assumed, but *pyBrill* should also run under *Windows*.

Several python modules are required:

- matplotlib
- pandas
- scipy

To check your installation, please open a command shell (we assume *bash*) and enter:

```
python3 --version
```

Python should show it's version number, i.e. Python is installed. Try:

```
python3  
import numpy, pandas, matplotlib, scipy
```

Leave Python (Ctrl+d under *Linux* , or exit() under *Windows*).

If modules are missing, it is recommended to build your own virtual python environment:

```
cd  
mkdir python_env  
python3 -m venv python_env
```

To switch to this environment, enter under *Linux* :

```
source /python_env/bin/activate
```

Under *Windows* enter:

```
python_env\Scripts\activate.bat
```

or

python_env\Scripts\activate.psl

To leave the python environment, enter:

deactivate

To install missing modules, activate your virtual python environment, as described above. Then enter:

pip3 install matplotlib

pip3 install pandas

pip3 install scipy

If you prefer ipython or idle to work with, you can install them the same way:

pip3 install ipython

pip3 install idle

I.2 FORTRAN

The brilliance calculations of *pyBrill* are executed in *Python*, the working horse of the other calculations is

urad_phase

written in *FORTRAN*.

If the open source *GNU* compiler *gfortran* is installed, the program can be compiled by the script

compile_urad_phase.sh

under *Linux* or

compile_urad_phase.bat

under *Windows*, respectively.

To transpose this tutorial for *Windows* users, replace

- the slashes in the path names by backslashes

- system variables like \$Path by %PATH%
- command *rm* by *del*

II Getting Started

First, we open a terminal and clone the whole package from Github to the directory *brill* :

```
git clone https://github.com/MicScheer/brill.git brill
```

We call the directory *brill* of *pyBrill* it's home directory and the sub-directory *stage* the working directory.

Since *urad_phase* is a standalone program, we can run it by hand, i.e. not deploying *pyBrill*. We change to the *working directory* and run *urad_phase*:

```
cd stage  
../bin/urad_phase.exe
```

The program should run and write some lines to the terminal.

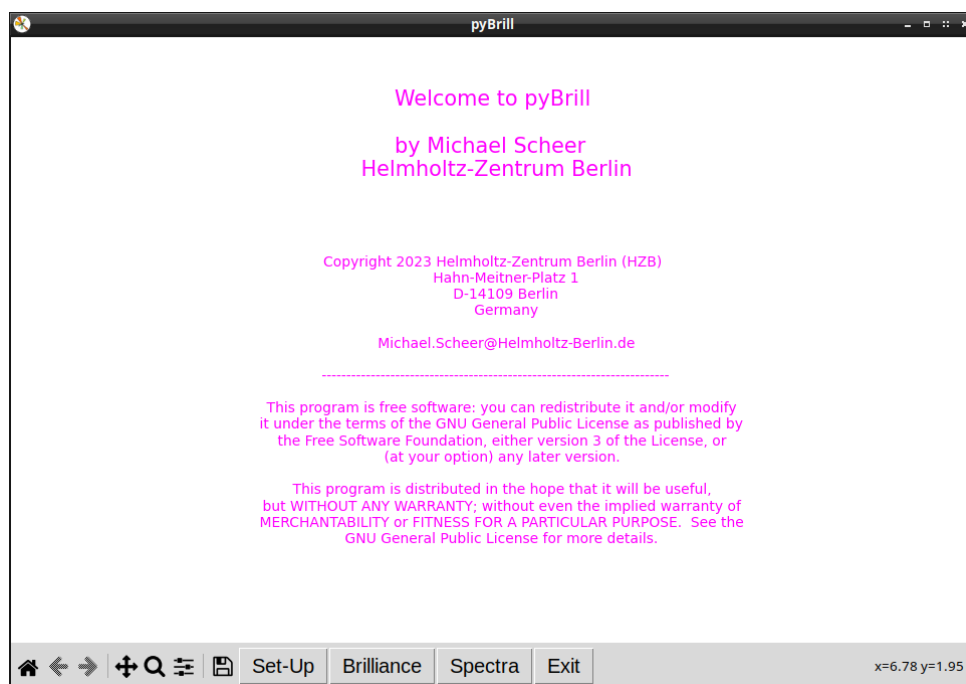


Fig. 1: Start Menu of pyBrill

Urad_phase reads the input data from a namelist file *urad_phase.nam*. This file can be edit by the user, but it also can be created by *pyBrill*, which has menus to set the variables, run *urad_phase*, and plots the results.

We start *pyBrill* and try it:

```
python3 -i ../python/pyBrill.py
```

The size and position of the window can be controlled by editing the first line of *ntupplot.cfg*

The other parameters are not relevant for *pyBrill*. An example of this file can be found in the sub-directory *input* of the home directory of *pyBrill*.

If present, the start-up file

```
ntupplot_startup.py
```

is evaluated and all command lines before the EOF line are executed at program start. The line

```
Load_Previous_Run()
```

loads the output of the previous run.

II.1 Brilliance Calculations

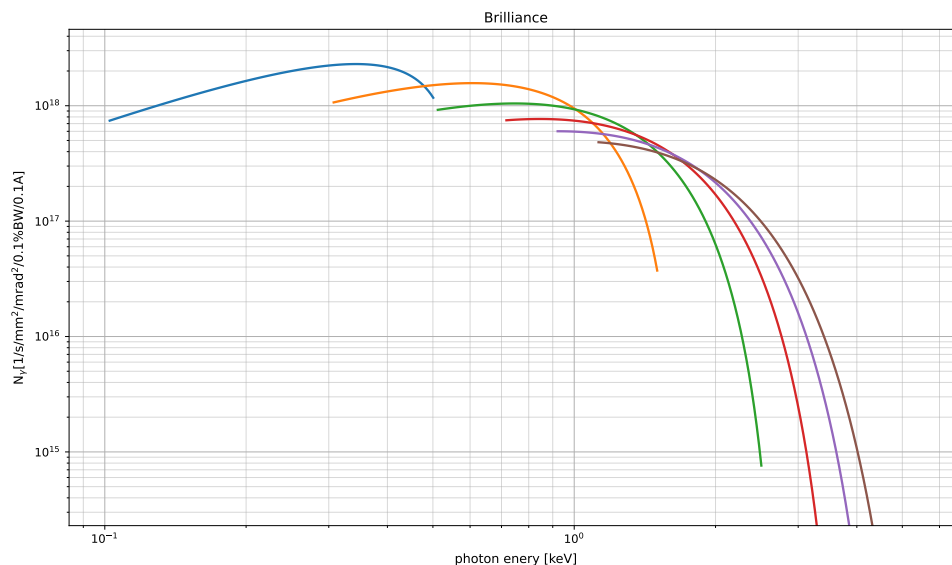


Fig. 2: Brilliance Plot

Press the button labeled *Brilliance* and plot the brilliance of the undulator. As you can see in the terminal, the plotted data are written to files. The plot itself is save as PDF-file.

To control the brilliance calculation, press the **Set-Up** button and check the menu items. The **spectra** item is not used the brilliance calculations. It is reserved for the spectra calculations with **urad_phase**.

The default values are set by **pyBrill** or read from a file named

.pybrill_last.dat

which is written when the **Exit** button of **pyBrill** is pressed. Note, if you leave **pyBrill** by entering "q" while the focus is on the output screen, the data are not stored.

II.2 Spectrum Calculations

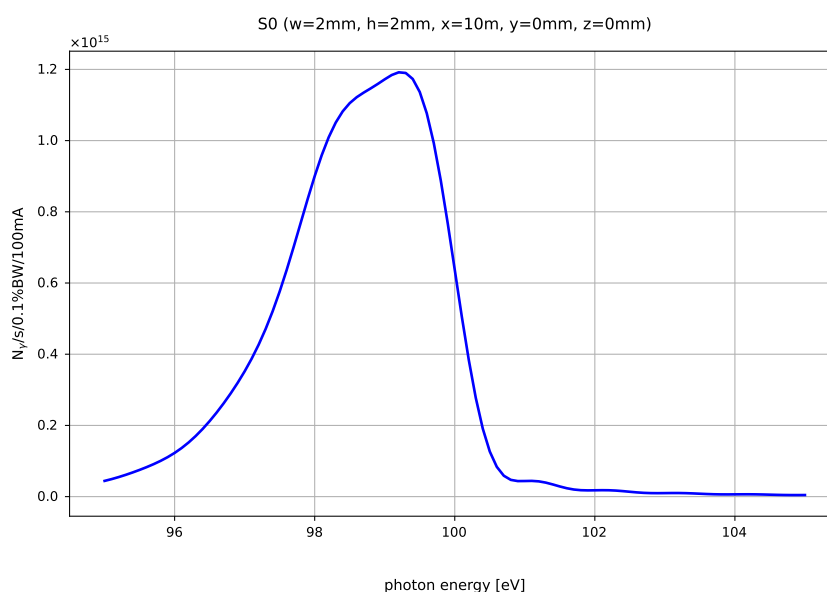


Fig. 3: Flux of Undulator

Press the **Spectra** button, select **Plot/Flux/Stokes/S0**. The flux of the Stokes parameter **S0** through the rectangular pinhole as calculated by **urad_phase** is plotted (Fig. 3).

Now, we select **Spectra/Plot/Distributions in Pinhole/Stokes/S0** and see the flux-density distribution of **S0**. By default the photon energy is selected to show the distribution for the photon energy with the maximum flux through the pinhole.

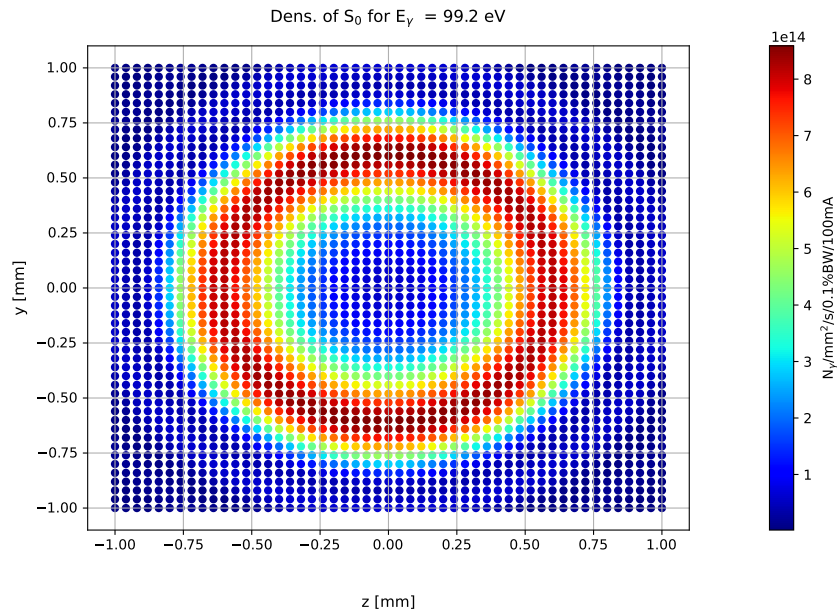


Fig. 4: Flux-density Distribution

To step through the photon energies, the arrows of the keyboard can be used. The photon energy can also be selected from the menu

Spectra/Plot/Distributions in Pinhole/Select E_{photon}

III Examples

There exists in the *input* directory a file for every given example.

You may copy this file to

.pybrill_last.dat

in the working directory, start *pyBrill* and trigger the calculations by pressing *Brilliance/Calculate* or *Spectra/Calculate*.¹ However, it is recommended to start with the first example and use *Set-Up* to proceed to the next example.

III.1 Basic Beam Electron Monte-Carlo

Before we start, we clean up the working directory to have a well defined, reproducible setting:

```
rm .pybrill_last.dat
rm urad_phase.bun
rm urad_phase.fld
rm urad_phase.flx
rm urad_phase.geo
rm urad_phase.nor
rm urad_phase.pin
rm ampgenpho.pho
rm ampgenpho.elc
rm urad_phase.run
```

We start *pyBrill*, press *Set-Up* , and set:

¹Note: The brilliance and spectra calculations are independent and must be triggered both if needed after changing the beam or undulator parameters.

Spectra Calculation	
Number of threads (<0: Use all cores)	-1
Nelec	1000
Each n_th electron is recorded	100
Monte-Carlo mode [0,1]	0
No random change for first e-	1
Arrange grid points on sphere [0,1]	0
Number of Photon Energies	11
Min. Photon Energy [eV]	695.0
Max. Photon Energy [eV]	705.0
X of Pinhole [mm]	10000.0
Y of Pinhole [mm]	0.0
Z of Pinhole [mm]	0.0
Width of Pinhole [mm]	1.0
Height of Pinhole [mm]	1.0
Number of hori. points	31
Number of vert. points	31
Tracking step size [mm]	0.2
Phase errors	0.0
Propagate radiation field back to origin	0
Long. Position of Plane [mm]	0.0
Width of Pinhole in Plane [mm]	0.1
Height of Pinhole in Plane [mm]	0.1
Number of vert. points in Plane	51
Number of hori. points in Plane	51
Random phase for e-	0
Global phase shift. 9999.: Hori. amplitude is zero for (PinX,PinY,PinZ)	9999.0
Calculate Wigner Distributions [-4,-3,-2,-1,0,1]	0
Wigner Distribution is not written to file [0,1]	0
Number of vert. angle steps	51
Vert. angle range	0.05
Number of hori. angle steps	51
Hori. angle range	0.05
Suppress splines for E-spread folding	0
Generate photons from Wigner-Distribution [-1,0,1]	0
Pattern of five bits to generate photons (Option iGenPho)	1111
Number of photons per electron to be generated	961
All grid points (zero) or Npho random grid points (not zero)	1
Number of E-spread steps for Wigner Dist.	1
Fix Seeds [0,1]	0
Close	

Fig. 5: *Set-Up/Spectra* for example 1

Nelec = 1000

Each n_th electron is recorded = 100

Number of Photon Energies = 11

Min. Photon Energy = 695.0

Max. Photon Energy = 705.0

Width of Pinhole = 1.0

Height of Pinhole = 1.0

Number of hori. points = 31

Number of vert. points = 31

We close the menu and click

Spectra/Calculate

We see, that the calculations take about one minutes, depending on your computer. To set the parameter for the recording of electrons to 100, reduces the size of the files

urad_phase.bun
ampgenpho.pho

and hence the time to run ***urad_phase*** significantly.

Spectra/Plot/Flux

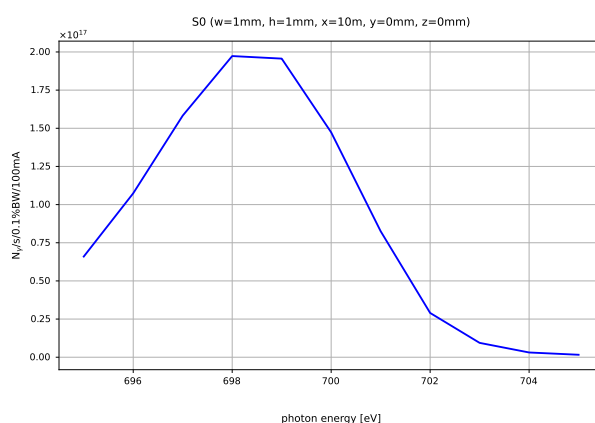


Fig. 6: ***Flux spectrum***

The flux spectrum we see, stems from 1000 electrons randomly chosen in there phase-space. Thus, the result will probably differ from the plots in this tutorial and change, when we rerun the calculations. However, in the set-up you find an option to fix the starting seed of the random number generator. This allows to reproduce results of a program run.

We have a look on the flux-density distribution in the pinhole (Fig. 6:

Spectra/Plot/Distribution in Pinhole/Stokes/S0

We change the plotting options by selecting:

Spectra/Plotting Options

and choosing 'inter' and plot the distribution again (Fig. 7).

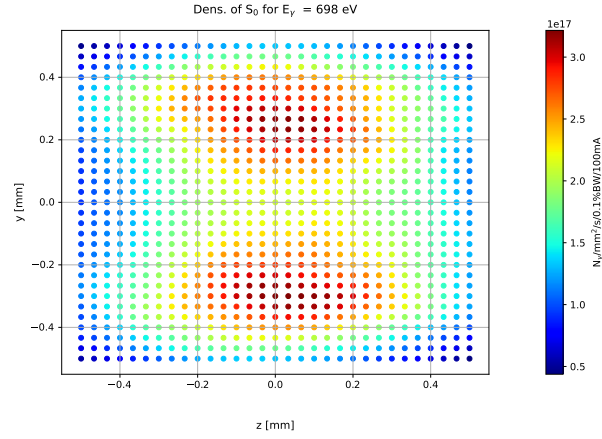


Fig. 7: *Flux spectrum, plotted with option 'scat2d'*

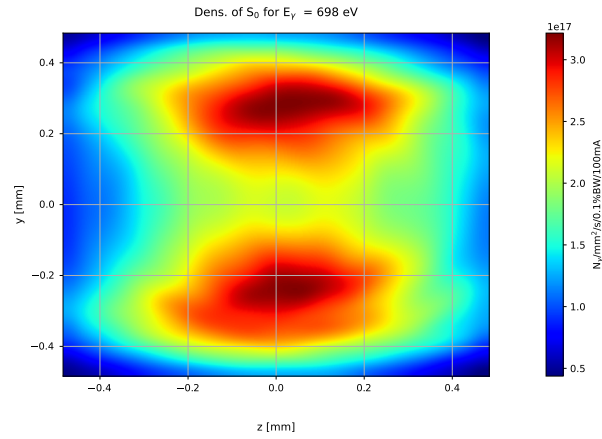


Fig. 8: *Flux spectrum, plotted with option 'inter'*

III.2 Combining Beam Electron and Observation Point Monte-Carlo

For a large number of electrons and many grid points, the calculation time can increase dramatically. To speed up the calculations of the flux through the pinhole, we select in *Set-Up/Spectra*

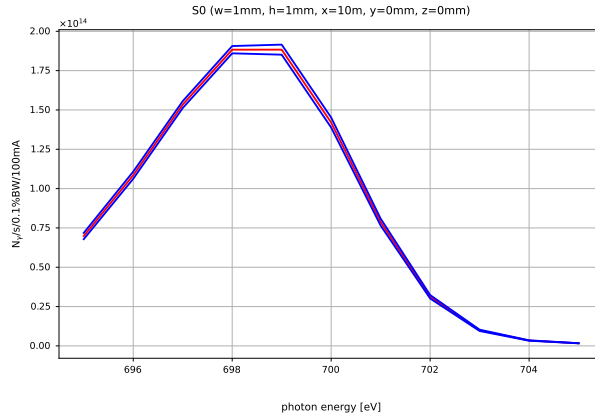
Nelec = 100000

Monte-Carlo mode = 1

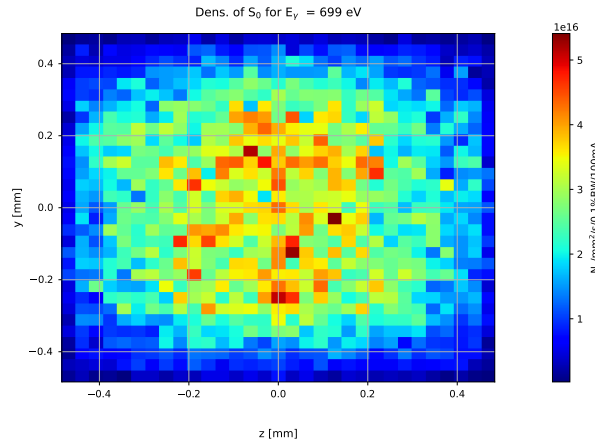
We close the menu and start the calculations.

Now, we only have to wait some seconds due to

Monte-Carlo mode = 1

Fig. 9: *Flux through pinhole with one sigma confidence level*

In this mode, for each electron a single observation point is randomly chosen in the pinhole. We see that flux spectrum shows only small statistical uncertainty (Fig. 9).

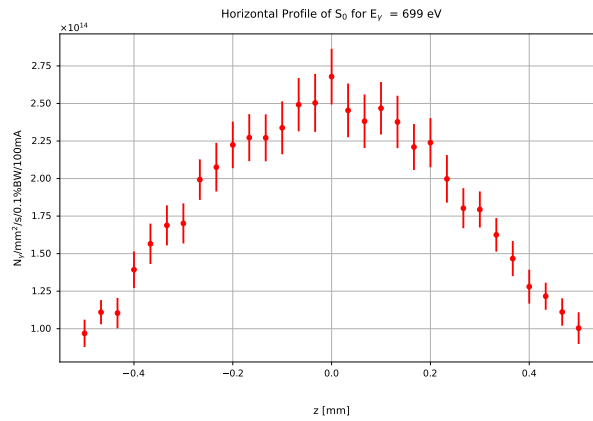
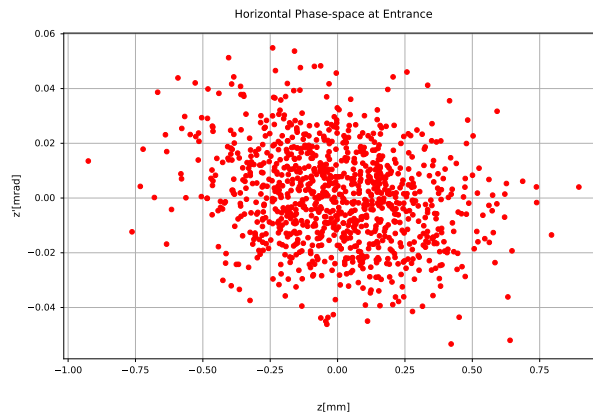
Fig. 10: *Flux-density in pinhole, plotted with option 'boxes'*

However, when we look at the distribution of the flux-density in the pinhole, we see clearly fluctuations due to this special Monte-Carlo mode (Fig. 10).

But in many cases, only the spectra of the flux, the flux-density, or the profiles of the distributions (Fig. 11) are of interest.

Also the reduced number of sampled electrons speeds up the calculations, since the I/O is reduced.

To check the phase-space a sample of 1000 electrons is certainly sufficient (Fig. 12).

Fig. 11: *Profile of flux-density in pinhole*Fig. 12: *Horizontal phase-space of beam electrons sample*

III.3 Back Propagation of the Radiation to the Source Plane

To calculate the source size, we propagate the radiation back to the source plane by selecting in *Set-Up/Spectra*

Nelec = 1
Monte-Carlo mode = 0
Number of Photons Energies = 1
Width of Pinhole = 2.0
Height of Pinhole = 2.0
Number of hori. points = 101
Number of vert. points = 101
Propagate radiation field back to origin = 1
Width of Pinhole in Plane = 0.1
Height of Pinhole in Plane = 0.1
Number of hori. points in Plane = 51
Number of vert. points in Plane = 51

and starting the calculations. Then we divide the canvas in the plotting options menu

Spectra/Plotting Options/

NxZones = 2

and plot the horizontal cut of the Stokes parameter S_0 in the observation and in the source plane:

Plot/Distribution in Pinhole/Cuts of Stokes/Hori. Cut of S_0

Plot/Distributions of Propagated Fields/Cuts of Stokes/Hori. Cut of S_0

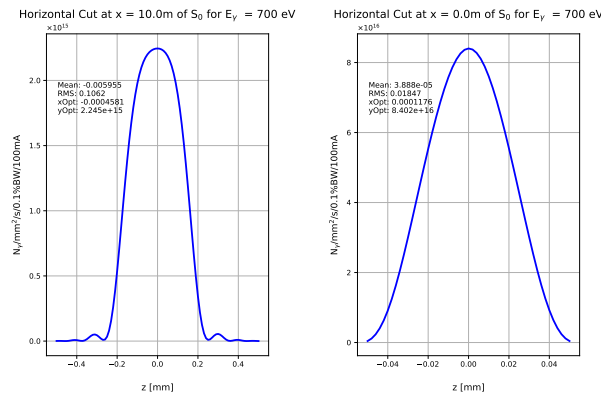


Fig. 13: *Horizontal cut of S_0 in the observation and in the source plane*

For the product of the RMS values of the two plots

$$\sigma_r \times \sigma_{r'} = \frac{\lambda}{4\pi}$$

we find

$$0.1062 \text{ mm} \times 0.01847 \text{ mm}/10 \text{ m} = 0.196 \text{ nm}$$

We have $\lambda = 1.77 \text{ nm}$ for 700 eV , thus

$$\frac{1.77 \text{ nm}}{4\pi} = 0.141 \text{ nm}$$

which underestimates the numerical value by a factor of about $\sqrt{2}$.

III.4 Generating Photons from Undulator Source for Ray-tracing

PyBrill uses the cross product of the calculated electric and magnetic fields to generate photons as input for ray-tracing codes.

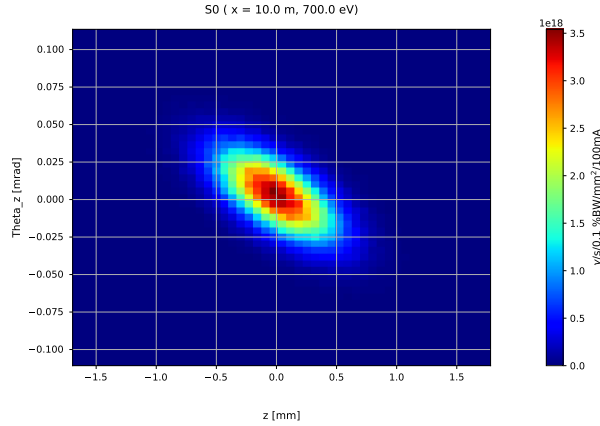


Fig. 14: *Horizontal phase-space of generated photons*

Due to the applied algorithm in this examples the photons are generated for an exact beam energy, i.e. without energy-spread.

To generate photons for electrons with energy-spread the program must sample the beam energy in discrete steps. The number of steps can be set in the set-up menu.

Note, if you run the program for another beam energy by hand, to see the effect of beam energy deviations, make shure that the undulator is set-up such that the fields are not adjusted according to the actual energy, as it is done, when a fixed harmonic is selected in the undulator set-up menu.

III.5 Wigner Distribution

To calculate the Wigner distribution from the fields in the source plane we select in *Set-Up/Spectra*

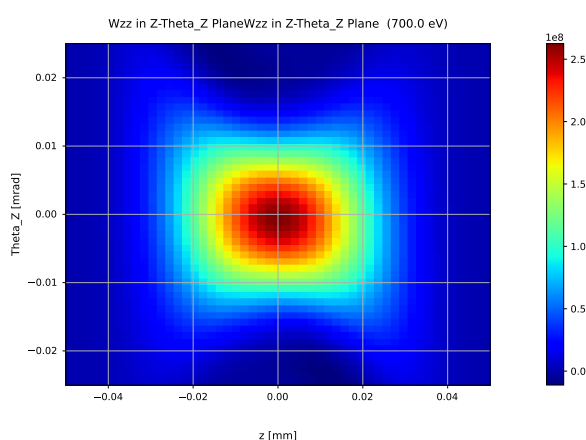


Fig. 15: *Wigner distribution in source plane*

Nelec = 1

Number of Photons Energies = 1

Width of Pinhole = 0.75

Height of Pinhole = 0.75

Number of hori. points = 101

Number of vert. points = 101

Propagate radiation field back to origin = 1

Width of Pinhole in Plane = 0.1

Height of Pinhole in Plane = 0.1

Number of hori. points in Plane = 51

Number of vert. points in Plane = 51

Calculate Wigner Distributions = -1

Number of vertical angle steps = 5

The line

Calculate Wigner Distributions = -1

triggers the Wigner calculation only for the Stokes parameter S_0 . The small number of vertical angle steps is used to speed up the calculations, since we are only interested in the distribution of the horizontal phase-space in this example (Fig. [15](#)).

The calculations of the Wigner distributions are still experimental. . .